

minimization routines, but which simply provides a common method for the initial bracketing of a minimum. In Chapter 7, the `Hashtable` object provides prerequisite methods to its derived classes `Hash` and `Mhash`, but one cannot say, “`Mhash` is-a `Hashtable`” in any meaningful way. An extreme example, in Chapter 6, is the base class `Gauleg18`, which does nothing except provide a bunch of constants for Gauss-Legendre integration to derived classes `Beta` and `Gamma`, both of which need them. Similarly, long lists of constants are provided to the routines `StepperDopr853` and `StepperRoss` in Chapter 17 by base classes to avoid cluttering the coding of the algorithms.

Partial Abstraction. Inheritance can be used in more complicated or situation-specific ways. For example, consider Chapter 4, where elementary quadrature rules such as `Trapzd` and `Midpnt` are used as building blocks to construct more elaborate quadrature algorithms. The key feature these simple rules share is a mechanism for adding more points to an existing approximation to an integral to get the “next” stage of refinement. This suggests deriving these objects from an abstract base class called `Quadrature`, which specifies that all objects derived from it must have a `next()` method. This is not a complete specification of a common is-a interface; it abstracts only one feature that turns out to be useful.

For example, in §4.6, the `Stiel` object invokes, in different situations, two different quadrature objects, `Trapzd` and `DERule`. These are not interchangeable. They have different constructor arguments and could not easily both be made `ZooAnimals` (as it were). `Stiel` of course knows about their differences. However, one of `Stiel`’s methods, `quad()`, doesn’t (and shouldn’t) know about these differences. It uses only the method `next()`, which exists, with different definitions, in both `Trapzd` and `DERule`.

While there are several different ways to deal with situations like this, an easy one is available once `Trapzd` and `DERule` have been given a common abstract base class `Quadrature` that contains nothing except a virtual interface to `next`. In a case like this, the base class is a minor design feature as far as the implementation of `Stiel` is concerned, almost an afterthought, rather than being the apex of a top-down design. As long as the usage is clear, there is nothing wrong with this.

Chapter 17, which discusses ordinary differential equations, has some even more complicated examples that combine inheritance and templating. We defer further discussion to there.

CITED REFERENCES AND FURTHER READING:

Stroustrup, B. 1997, *The C++ Programming Language*, 3rd ed. (Reading, MA: Addison-Wesley).[1]

Lippman, S.B., Lajoie, J., and Moo, B.E. 2005, *C++ Primer*, 4th ed. (Boston: Addison-Wesley).[2]

Keogh, J., and Giannini, M. 2004, *OOP Demystified* (Emeryville, CA: McGraw-Hill/Osborne).[3]

Cline, M., Lomow, G., and Girou, M. 1999, *C++ FAQs*, 2nd ed. (Boston: Addison-Wesley).[4]

1.4 Vector and Matrix Objects

The C++ Standard Library [1] includes a perfectly good `vector<>` template class. About the only criticism that one can make of it is that it is so feature-rich

that some compiler vendors neglect to squeeze the last little bit of performance out of its most elementary operations, for example returning an element by its subscript. That performance is extremely important in scientific applications; its occasional absence in C++ compilers is a main reason that many scientists still (as we write) program in C, or even in Fortran!

Also included in the C++ Standard Library is the class `valarray<>`. At one time, this was supposed to be a vector-like class that was optimized for numerical computation, including some features associated with matrices and multidimensional arrays. However, as reported by one participant,

The `valarray` classes were not designed very well. In fact, nobody tried to determine whether the final specification worked. This happened because nobody felt “responsible” for these classes. The people who introduced `valarrays` to the C++ standard library left the committee a long time before the standard was finished. [1]

The result of this history is that C++, at least now, has a good (but not always reliably optimized) class for vectors and no dependable class at all for matrices or higher-dimensional arrays. What to do? We will adopt a strategy that emphasizes flexibility and assumes only a minimal set of properties for vectors and matrices. We will then provide our own, basic, classes for vectors and matrices. For most compilers, these are at least as efficient as `vector<>` and other vector and matrix classes in common use. But if, for you, they’re not, then it is easy to change to a different set of classes, as we will explain.

1.4.1 Typedefs

Flexibility is achieved by having several layers of `typedef` type-indirection, resolved at compile time so that there is no run-time performance penalty. The first level of type-indirection, not just for vectors and matrices but for virtually all variables, is that we use user-defined type names instead of C++ fundamental types. These are defined in `nr3.h`. If you ever encounter a compiler with peculiar built-in types, these definitions are the “hook” for making any necessary changes. The complete list of such definitions is

<i>NR Type</i>	<i>Usual Definition</i>	<i>Intent</i>
Char	<code>char</code>	8-bit signed integer
Uchar	<code>unsigned char</code>	8-bit unsigned integer
Int	<code>int</code>	32-bit signed integer
Uint	<code>unsigned int</code>	32-bit unsigned integer
Llong	<code>long long int</code>	64-bit signed integer
Ullong	<code>unsigned long long int</code>	64-bit unsigned integer
Doub	<code>double</code>	64-bit floating point
Ldoub	<code>long double</code>	[reserved for future use]
Complex	<code>complex<double></code>	2 × 64-bit floating complex
Bool	<code>bool</code>	true or false

An example of when you might need to change the typedefs in `nr3.h` is if your compiler’s `int` is not 32 bits, or if it doesn’t recognize the type `long long int`.

You might need to substitute vendor-specific types like (in the case of Microsoft) `__int32` and `__int64`.

The second level of type-indirection returns us to the discussion of vectors and matrices. The vector and matrix types that appear in *Numerical Recipes* source code are as follows. Vectors: `VecInt`, `VecUInt`, `VecChar`, `VecUchar`, `VecCharp`, `VecLlong`, `VecUllong`, `VecDoub`, `VecDoubp`, `VecComplex`, and `VecBool`. Matrices: `MatInt`, `MatUInt`, `MatChar`, `MatUchar`, `MatLlong`, `MatUllong`, `MatDoub`, `MatComplex`, and `MatBool`. These should all be understandable, semantically, as vectors and matrices whose elements are the corresponding user-defined types, above. Those ending in a “p” have elements that are pointers, e.g., `VecCharp` is a vector of pointers to `char`, that is, `char*`. If you are wondering why the above list is not combinatorially complete, it is because we don’t happen to use all possible combinations of `Vec`, `Mat`, fundamental type, and pointer in this book. You can add further analogous types as you need them.

Wait, there’s more! For every vector and matrix type above, we also define types with the same names plus one of the suffixes “_I”, “_O”, and “_IO”, for example `VecDoub_IO`. We use these suffixed types for specifying argument types in function definitions. The meaning, respectively, is that the argument is “input”, “output”, or “both input and output”.^{*} The `_I` types are automatically defined to be `const`. We discuss this further in §1.5.2 under the topic of `const` correctness.

It may seem capricious for us to define such a long list of types when a much smaller number of templated types would do. The rationale is flexibility: You have a hook into redefining each and every one of the types individually, according to your needs for program efficiency, local coding standards, `const`-correctness, or whatever. In fact, in `nr3.h`, all these types *are* typedef’d to one vector and one matrix class, along the following lines:

```
typedef NRvector<Int> VecInt, VecInt_0, VecInt_IO;
typedef const NRvector<Int> VecInt_I;
...
typedef NRvector<Doub> VecDoub, VecDoub_0, VecDoub_IO;
typedef const NRvector<Doub> VecDoub_I;
...
typedef NRmatrix<Int> MatInt, MatInt_0, MatInt_IO;
typedef const NRmatrix<Int> MatInt_I;
...
typedef NRmatrix<Doub> MatDoub, MatDoub_0, MatDoub_IO;
typedef const NRmatrix<Doub> MatDoub_I;
...
```

So (flexibility, again) you can change the definition of one particular type, like `VecDoub`, or else you can change the implementation of all vectors by changing the definition of `NRvector<>`. Or, you can just leave things the way we have them in `nr3.h`. That ought to work fine in 99.9% of all applications.

1.4.2 Required Methods for Vector and Matrix Classes

The important thing about the vector and matrix classes is not what names they are typedef’d to, but what methods are assumed for them (and are provided in the `NRvector` and `NRmatrix` template classes). For vectors, the assumed methods are a

^{*}This is a bit of history, and derives from Fortran 90’s very useful `INTENT` attributes.

subset of those in the C++ Standard Library `vector<>` class. If `v` is a vector of type `NRvector<T>`, then we assume the methods:

<code>v()</code>	Constructor, zero-length vector.
<code>v(Int n)</code>	Constructor, vector of length <code>n</code> .
<code>v(Int n, const T &a)</code>	Constructor, initialize all elements to the value <code>a</code> .
<code>v(Int n, const T *a)</code>	Constructor, initialize elements to values in a C-style array, <code>a[0]</code> , <code>a[1]</code> , ...
<code>v(const NRvector &rhs)</code>	Copy constructor.
<code>v.size()</code>	Returns number of elements in <code>v</code> .
<code>v.resize(Int newn)</code>	Resizes <code>v</code> to size <code>newn</code> . We do not assume that contents are preserved.
<code>v.assign(Int newn, const T &a)</code>	Resize <code>v</code> to size <code>newn</code> , and set all elements to the value <code>a</code> .
<code>v[Int i]</code>	Element of <code>v</code> by subscript, either an l-value and an r-value.
<code>v = rhs</code>	Assignment operator. Resizes <code>v</code> if necessary and makes it a copy of the vector <code>rhs</code> .
<code>typedef T value_type;</code>	Makes <code>T</code> available externally (useful in templated functions or classes).

As we will discuss later in more detail, you can use any vector class you like with *Numerical Recipes*, as long as it provides the above basic functionality. For example, a brute force way to use the C++ Standard Library `vector<>` class instead of `NRvector` is by the preprocessor directive

```
#define NRvector vector
```

(In fact, there is a compiler switch, `_USESTDVECTOR_`, in the file `nr3.h` that will do just this.)

The methods for matrices are closely analogous. If `vv` is a matrix of type `NRmatrix<T>`, then we assume the methods:

<code>vv()</code>	Constructor, zero-length vector.
<code>vv(Int n, Int m)</code>	Constructor, $n \times m$ matrix.
<code>vv(Int n, Int m, const T &a)</code>	Constructor, initialize all elements to the value <code>a</code> .
<code>vv(Int n, Int m, const T *a)</code>	Constructor, initialize elements by rows to the values in a C-style array.
<code>vv(const NRmatrix &rhs)</code>	Copy constructor.
<code>vv.nrows()</code>	Returns number of rows <code>n</code> .
<code>vv.ncols()</code>	Returns number of columns <code>m</code> .
<code>vv.resize(Int newn, Int newm)</code>	Resizes <code>vv</code> to $\text{newn} \times \text{newm}$. We do not assume that contents are preserved.
<code>vv.assign(Int newn, Int newm, const T &a)</code>	Resizes <code>vv</code> to $\text{newn} \times \text{newm}$, and sets all elements to the value <code>a</code> .
<code>vv[Int i]</code>	Return a pointer to the first element in row <code>i</code> (not often used by itself).
<code>v[Int i][Int j]</code>	Element of <code>vv</code> by subscript, either an l-value and an r-value.
<code>vv = rhs</code>	Assignment operator. Resizes <code>vv</code> if necessary and makes it a copy of the matrix <code>rhs</code> .
<code>typedef T value_type;</code>	Makes <code>T</code> available externally.

For more precise specifications, see §1.4.3.

There is one additional property that we assume of the vector and matrix classes, namely that all of an object's elements are stored in sequential order. For a vector, this means that its elements can be addressed by pointer arithmetic relative to the first element. For example, if we have

```
VecDoub a(100);
Doub *b = &a[0];
```

then `a[i]` and `b[i]` reference the same element, both as an l-value and as an r-value. This capability is sometimes important for inner-loop efficiency, and it is also useful for interfacing with legacy code that can handle `Doub*` arrays, but not `VecDoub` vectors. Although the original C++ Standard Library did not guarantee this behavior, all known implementations of it do so, and the behavior is now required by an amendment to the standard [2].

For matrices, we analogously assume that storage is by rows within a single sequential block so that, for example,

```
Int n=97, m=103;
MatDoub a(n,m);
Doub *b = &a[0][0];
```

implies that `a[i][j]` and `b[m*i+j]` are equivalent.

A few of our routines need the capability of taking as an argument either a vector or else one row of a matrix. For simplicity, we usually code this using overloading, as, e.g.,

```
void someroutine(Doub *v, Int m) {           Version for a matrix row.
    ...
}
inline void someroutine(VecDoub &v) {       Version for a vector.
    someroutine(&v[0],v.size());
}
```

For a vector `v`, a call looks like `someroutine(v)`, while for row `i` of a matrix `vv` it is `someroutine(&vv[i][0],vv.ncols())`. While the simpler argument `vv[i]` would in fact work in our implementation of `NRmatrix`, it might not work in some other matrix class that guarantees sequential storage but has the return type for a single subscript different from `T*`.

1.4.3 Implementations in `nr3.h`

For reference, here is a complete declaration of `NRvector`.

```
template <class T>
class NRvector {
private:
    int nn;                               Size of array, indices 0..nn-1.
    T *v;                                 Pointer to data array.
public:
    NRvector();                            Default constructor.
    explicit NRvector(int n);              Construct vector of size n.
    NRvector(int n, const T &a);           Initialize to constant value a.
    NRvector(int n, const T *a);           Initialize to values in C-style array a.
    NRvector(const NRvector &rhs);         Copy constructor.
    NRvector & operator=(const NRvector &rhs); Assignment operator.
    typedef T value_type;                 Make T available.
    inline T & operator[](const int i);    Return element number i.
    inline const T & operator[](const int i) const; const version.
    inline int size() const;              Return size of vector.
    void resize(int newn);                 Resize, losing contents.
    void assign(int newn, const T &a);     Resize and assign a to every element.
    ~NRvector();                           Destructor.
};
```

The implementations are straightforward and can be found in the file `nr3.h`. The only issues requiring finesse are the consistent treatment of zero-length vectors and the avoidance of unnecessary resize operations.

A complete declaration of `NRmatrix` is

<pre>template <class T> class NRmatrix { private: int nn; int mm; T **v; public: NRmatrix(); NRmatrix(int n, int m); NRmatrix(int n, int m, const T &a); NRmatrix(int n, int m, const T *a); NRmatrix(const NRmatrix &rhs); NRmatrix & operator=(const NRmatrix &rhs); typedef T value_type; inline T* operator[](const int i); inline const T* operator[](const int i) const; inline int nrow() const; inline int ncol() const; void resize(int newn, int newm); void assign(int newn, int newm, const T &a); ~NRmatrix(); };</pre>	Number of rows and columns. Index range is 0..<code>nn</code>-1, 0..<code>mm</code>-1. Storage for data. Default constructor. Construct $n \times m$ matrix. Initialize to constant value <code>a</code>. Initialize to values in C-style array <code>a</code>. Copy constructor. Assignment operator. Make <code>T</code> available. Subscripting: pointer to row <code>i</code>. const version. Return number of rows. Return number of columns. Resize, losing contents. Resize and assign <code>a</code> to every element. Destructor.
--	---

A couple of implementation details in `NRmatrix` are worth commenting on. The private variable `**v` points not to the data but rather to an array of pointers to the data rows. Memory allocation of this array is separate from the allocation of space for the actual data. The data space is allocated as a single block, not separately for each row. For matrices of zero size, we have to account for the separate possibilities that there are zero rows, or that there are a finite number of rows, but each with zero columns. So, for example, one of the constructors looks like this:

```
template <class T>
NRmatrix<T>::NRmatrix(int n, int m) : nn(n), mm(m), v(n>0 ? new T*[n] : NULL)
{
    int i,nel=m*n;
    if (v) v[0] = nel>0 ? new T[nel] : NULL;
    for (i=1;i<n;i++) v[i] = v[i-1] + m;
}
```

Finally, it matters *a lot* whether your compiler honors the `inline` directives in `NRvector` and `NRmatrix` above. If it doesn't, then you may be doing full function calls, saving and restoring context within the processor, every time you address a vector or matrix element. This is tantamount to making C++ useless for most numerical computing! Luckily, as we write, the most commonly used compilers are all "honorable" in this respect.

CITED REFERENCES AND FURTHER READING:

- Josuttis, N.M. 1999, *The C++ Standard Library: A Tutorial and Reference* (Boston: Addison-Wesley).[1]
- International Standardization Organization 2003, *Technical Corrigendum ISO 14882:2003*. [2]

1.5 Some Further Conventions and Capabilities

We collect in this section some further explanation of C++ language capabilities and how we use them in this book.

1.5.1 Error and Exception Handling

We already mentioned that we code error conditions with simple `throw` statements, like this

```
throw("error foo in routine bah");
```

If you are programming in an environment that has a defined set of error classes, and you want to use them, then you'll need to change these lines in the routines that you use. Alternatively, without any additional machinery, you can choose between a couple of different, useful behaviors just by making small changes in `nr3.h`.

By default, `nr3.h` redefines `throw()` by a preprocessor macro,

```
#define throw(message) \
    {printf("ERROR: %s\n in file %s at line %d\n", \
        message, __FILE__, __LINE__); \
      exit(1);};
```

This uses standard ANSI C features, also present in C++, to print the source code file name and line number at which the error occurs. It is inelegant, but perfectly functional.

Somewhat more functional, and definitely more elegant, is to set `nr3.h`'s compiler switch `_USENRErrorCLASS_`, which instead substitutes the following code:

```
struct NRError {
    char *message;
    char *file;
    int line;
    NRError(char *m, char *f, int l) : message(m), file(f), line(l) {}
};

void NRcatch(NRError err) {
    printf("ERROR: %s\n      in file %s at line %d\n",
        err.message, err.file, err.line);
    exit(1);
}

#define throw(message) throw(NRError(message, __FILE__, __LINE__));
```

Now you have a (rudimentary) error class, `NRError`, available. You use it by putting a `try...catch` control structure at any desired point (or points) in your code, for example (§2.9),

```
...
try {
    Cholesky achol(a);
}
catch (NRError err) {
    NRcatch(err);           Executed if Cholesky throws an exception.
}
```

As shown, the use of the `NRcatch` function above simply mimics the behavior of the previous macro in the global context. But you don't have to use `NRcatch` at all: You

can substitute any code that you want for the body of the `catch` statement. If you want to distinguish between different kinds of exceptions that may be thrown, you can use the information returned in `err`. We'll let you figure this out yourself. And of course you are welcome to add more complicated error classes to your own copy of `nr3.h`.

1.5.2 Const Correctness

Few topics in discussions about C++ evoke more heat than questions about the keyword `const`. We are firm believers in using `const` wherever possible, to achieve what is called “const correctness.” Many coding errors are automatically trapped by the compiler if you have qualified identifiers that should not change with `const` when they are declared. Also, using `const` makes your code much more readable: When you see `const` in front of an argument to a function, you know immediately that the function will not modify the object. Conversely, if `const` is absent, you should be able to count on the object being changed somewhere.

We are such strong `const` believers that we insert `const` even where it is theoretically redundant: If an argument is passed *by value* to a function, then the function makes a copy of it. Even if this copy is modified by the function, the original value is unchanged after the function exits. While this allows you to change, with impunity, the values of arguments that have been passed by value, this usage is error-prone and hard to read. If your intention in passing something by value is that it is an input variable only, then make it clear. So we declare a function $f(x)$ as, for example,

```
Doub f(const Doub x);
```

If in the function you want to use a local variable that is initialized to `x` but then gets changed, define a new quantity — don't use `x`. If you put `const` in the declaration, the compiler will not let you get this wrong.

Using `const` in your function arguments makes your function more general: Calling a function that expects a `const` argument with a non-`const` variable involves a “trivial” conversion. But trying to pass a `const` quantity to a non-`const` argument is an error.

A final reason for using `const` is that it allows certain user-defined conversions to be made. As discussed in [1], this can be useful if you want to use *Numerical Recipes* routines with another matrix/vector class library.

We now need to elaborate on what exactly `const` does for a nonsimple type such as a class that is an argument of a function. Basically, it guarantees that the object is not modified by the function. In other words, the object's data members are unchanged. But if a data member is a *pointer* to some data, and the data itself is not a member variable, then *the data can be changed* even though the pointer cannot be.

Let's look at the implications of this for a function `f` that takes an `NRvector<Doub>` argument `a`. To avoid unnecessary copying, we always pass vectors and matrices by reference. Consider the difference between declaring the argument of a function with and without `const`:

```
void f(NRvector<Doub> &a)      versus      void f(const NRvector<Doub> &a)
```

The `const` version promises that `f` does not modify the data members of `a`. But a statement like

```
a[i] = 4.;
```